

Le socle est bon. Il manque deux gardes et une couche.

Trois audits successifs — positionnement des pages, modèle de sécurité, revue du dual-role — convergent vers la même conclusion. Ce projet n'a pas besoin d'une refonte : il a besoin de **quatre ajouts précis** qui rendent cohérent ce qui est déjà construit. Voici le modèle cible, exprimé comme une extension de l'existant.

41	16	2	1
modules de logique partagée déjà extraits	gardes et prédicats métier réutilisables	familles de gardes manquantes	couche absente (périmètre)

Ce qui constitue déjà le socle

L'inventaire corrige une impression : ce n'est pas un projet à sécuriser après coup. La logique métier est *déjà* sortie des routes et centralisée — c'est exactement la structure qui rend les ajouts ci-après peu coûteux. Le modèle unifié conserve tout cela sans exception.

Logique hors des routes 41 modules dans <code>src/lib</code> : devis, jalons, garants, risque, âge, fraude, hash. Les routes orchestrent, elles ne décident pas.	Espace mission neutre Les 9 étapes du cycle vivent sous <code>/missions/[id]/*</code> , partagées par les deux parties — aucune duplication par rôle.
Multi-profil natif <code>Profile[]</code> nommés et indépendants, avec domaine, tarifs, zone, garants et déclarations propres à chacun.	Fonds jamais détenus La plateforme instruit un PSP agréé. Aucun solde interne, aucun portefeuille — position juridique fondatrice.

Chaînage par hash

Déclarations, contrats et journal d'audit chaînés — toute réécriture rétroactive est détectable.

Séparation des pouvoirs

Quatre rôles admin distincts, justification obligatoire, journal append-only.

Détection de fraude

Collusion, doublons de comptes, empreinte perceptuelle des pièces, heuristiques — déjà branchées côté admin.

Identité vérifiée une fois

kycStatus porté par le compte, valable quelle que soit la face — prêt pour le dual-role.

Le modèle : quatre familles de gardes

Toute la sécurité applicative se ramène à quatre questions, posées dans cet ordre. Le projet répond déjà solidement aux deux premières et à une partie de la troisième. Les deux dernières sont les manques — et ce sont elles qui conditionnent l'ouverture du dual-role.

Existe et fonctionne

À construire

1· Qui appelle ?

EN PLACE

Session JWT revalidée en base à chaque requête : un compte supprimé, suspendu ou banni perd sa session immédiatement, sans attendre l'expiration du jeton. Le rôle et le statut admin sont rafraîchis au même moment.

```
src/auth.ts · callback jwt → null si compte absent ou inactif
```

2· A-t-il le bon rôle et le bon statut ?

EN PLACE

Deux gardes partagés, appliqués systématiquement : `requireAdmin()` avec granularité par `adminRole`, et `requireVerifiedKyc()` qui conditionne les deux seules actions gâtées par le KYC — publier une mission et candidater.

```
adminGuard.ts · kycGuard.ts
```

3· A-t-il le droit sur *cette* ressource ?

MANQUANTE

C'est le trou principal. Il existe des gardes d'*éligibilité*

(`checkCandidatureEligibility`) mais aucun garde de **propriété**. Chaque route improvise : certaines filtrent correctement dans le `where`, trois ne filtrent pas du tout — d'où les failles F-01, F-02 et F-03 de l'audit de sécurité.

à créer · `src/lib/resource-guard.ts`

4 · L'opération est-elle cohérente en elle-même ?

MANQUANTE

La question qu'aucune couche ne pose aujourd'hui : indépendamment des droits de l'appelant, l'écriture demandée produit-elle un état valide ? Un contrat entre une personne et elle-même est refusé non pas faute de droits, mais parce qu'il n'a **pas de sens**. C'est cette famille que le dual-role rend indispensable.

à créer · `src/lib/invariants.ts`

Pourquoi la quatrième famille n'est pas optionnelle

Le dual-role rend aveugle la détection de collusion existante

Le projet détecte déjà la collusion : `detectCollusionSuspicion()` lève une alerte quand un payeur et un bénéficiaire partagent la même empreinte d'appareil sur au moins trois transactions à montants ronds. C'est un contrôle **détectif**, et il suppose **deux comptes distincts**.

En dual-role, l'auto-attribution ne nécessite plus deux comptes : le payeur et le bénéficiaire sont littéralement le même utilisateur. Le motif que la détection cherche — deux identités liées par un appareil — n'existe plus. Et même si l'alerte finissait par se déclencher, elle arriverait **après trois passages de fonds réels par le PSP**.

C'est la démonstration que la prévention ne peut pas être remplacée par la détection ici : le contrôle doit refuser l'écriture, pas la signaler plus tard.

Et la faille est déjà ouverte, avant même le dual-role

Recherche exhaustive du dépôt : **aucune comparaison entre le commanditaire d'une mission et l'auteur d'une candidature n'existe**. La route de candidature contrôle le KYC, le statut de la mission, les garants, l'assurance, les doublons et les rounds — jamais l'identité croisée. Aujourd'hui, seule l'interface masque le chemin

```
;l'API l'accepte.
```

Les deux modules à écrire

Environ soixante lignes au total. Ils suivent exactement la convention déjà en vigueur dans `src/lib` : logique pure, testable isolément, importée par les routes.

Famille 3 — la propriété

La condition entre dans le `where`. Une ressource sans droit est indistinguishable d'une ressource absente.

```
// src/lib/resource-guard.ts

export async function requireMissionParty(missionId: string, userId: string) {
  const mission = await prisma.mission.findFirst({
    where: { id: missionId, OR: [
      { clientId: userId },
      { proposals: { some: { providerId: userId } } },
    ]},
    include: { proposals: { select: { providerId: true } } },
  });
  if (!mission) return { ok: false, status: 404 } as const;

  // Sous dual-role les deux branches ne sont plus exclusives :
  // on qualifie explicitement au lieu de deduire l'un de l'autre.
  const isClient = mission.clientId === userId;
  const isProvider = mission.proposals.some((p) => p.providerId === userId);

  return { ok: true, mission, isClient, isProvider } as const;
}
```

Famille 4 — les invariants

Refuse une écriture incohérente quels que soient les droits de l'appelant.

```
// src/lib/invariants.ts

// Une personne ne peut pas etre les deux parties d'un meme contrat.
// Vaut aujourd'hui (role unique) et demain (dual-role) : c'est une regle
// sur la donnee, pas sur le role de l'appelant.
export function assertNoSelfDealing(clientId: string, providerId: string) {
  return clientId !== providerId;
}
```

```

}

// Invariant de lecture : si les deux faces sont vraies sur une meme mission,
// une donnee invalide existe deja – aucune decision d'autorisation n'est sure.
export function assertPartyExclusive(isClient: boolean, isProvider: boolean) {
  return !(isClient && isProvider);
}

```

La route type, avec les quatre familles

Ordre invariable. Toute route à identifiant doit être lisible sur ce gabarit.

```

export async function POST(req, { params }) {
  // 1 – identite
  const session = await auth();
  if (!session?.user) return json({ error: "unauthenticated" }, 401);
  const userId = session.user.id;

  // 2 – role / statut
  const kyc = await requireVerifiedKyc();
  if (!kyc.ok) return json({ error: kyc.error }, kyc.status);

  // 3 – propriete de la ressource visee
  const guard = await requireMissionParty(params.id, userId);
  if (!guard.ok) return json({ error: "not_found" }, 404);

  // 4 – coherence de l'ecriture demandee
  if (!assertNoSelfDealing(guard.mission.clientId, userId)) {
    return json({ error: "self_dealing_forbidden" }, 403);
  }

  // ... logique metier (deja dans src/lib)
}

```

Le dual-role, exprimé dans ce modèle

Une fois les quatre familles en place, le dual-role ne demande presque aucun code : il devient une question d'affichage sur des données que la base distingue déjà.

La face se dérive, elle ne se stocke pas

User.missionsClient et User.proposalsSoumises sont deux relations distinctes sur

la meme personne : la base sait deja repondre « ce que je publie » contre « ce a quoi je candidate ». Le `requireMissionParty()` ci-dessus renvoie déjà `isClient` et `isProvider` — c'est la face, calculée par mission.

Aucun `active_role` à synchroniser entre le navigateur, une table de sessions et le jeton. Le « mode » devient un filtre de lecture, pas un état.

UserRole reste la filière métier, pas la face

Artisan, manoeuvre, expert BTP, expert digital décrivent *ce que je sais faire* — pas de quel côté du marché je me place. Cette distinction résout au passage l'ambiguïté du contrôle d'âge : la barrière A13 porte sur le **profil prestataire**, jamais sur le compte, donc un compte à double face ne l'esquive pas et ne la subit pas hors sujet.

Les notes se calculent par face, sans migration

Review porte déjà `missionId`, `authorId` et `targetId` : la face de l'évalué se déduit de la mission — noté comme client si la mission était la sienne, comme prestataire sinon. Deux agrégats de lecture, aucune colonne dénormalisée à maintenir en cohérence.

Le portefeuille reste une vue, jamais un solde

L'écran peut afficher « encaissé » et « dépensé » côte à côte : c'est une agrégation sur les opérations PSP confirmées. Ce qu'il ne devient jamais, c'est un solde détenu par la plateforme — la distinction est invisible à l'écran et déterminante juridiquement.

Séquence en trois vagues

Les portes ne sont pas décoratives : la vague 2 ne peut pas commencer avant que la vague 1 soit close, sous peine de transformer une faille en fonctionnalité.

Vague 1 — Fermer ce qui est ouvert

PRÉALABLE AU DUAL-ROLE

- Créer `src/lib/invariants.ts` et brancher `assertNoSelfDealing` sur la candidature et la génération de contrat.
- Créer `src/lib/resource-guard.ts` ; fermer F-01 (candidatures lisibles par tous) et F-

02 (signatures lisibles par tous).

- Fermer F-03 : exiger une mission liant émetteur et destinataire avant tout relais de signal visio.
- Créer `middleware.ts` : plafond par IP et en-têtes de sécurité — la couche périmètre, aujourd'hui absente.

Vague 2 — Rendre la régression difficile

ENSUITE

- Rebrancher toutes les routes à identifiant sur les gardes partagés, en remplacement des contrôles improvisés.
- Pour chaque route à identifiant : un test « utilisateur tiers → 404 ». C'est ce test qui manque partout.